

CLBIPP

Closed-Loop Battery Intelligence & Pricing Platform

Module 2 — Decision & Pricing Engine

Compiled Specification

Version 1.0

1. Purpose

Take a used Li-ion battery — typically from internal warehouse stock, possibly from external suppliers — and produce, in seconds:

1. The optimal pathway: Reuse, Refurbish, or Recycle
2. A net economic value (₹)
3. For external buys, a configurable purchase price band
4. A transparent breakdown explaining the decision

The engine is **deterministic, rule-based, configurable, and auditable**. No ML in the decision path. Every output is reproducible from logged inputs and config version.

2. Sourcing Context

The platform handles two inflow types using the same engine:

- **Internal:** retired fleet batteries pulled into warehouses. No purchase price; output is recovered value.
- **External:** batteries bought from third parties (fleets, scrap dealers, OEMs). Engine outputs a purchase price band.

The math is identical; purchase cost is zero for internal inflows. Margin tiers operate as a hurdle rate for internal stock and as a price-band derivation for external.

3. Layered Architecture

The engine processes a battery through six sequential layers. Each layer can short-circuit to RECYCLE or HOLD; each produces structured output for the next.

Layer 0	Intake & Validation
Layer 1	Physical Damage Scoring (visual / external)
Layer 2	BMS Safety Assessment (electrical / internal)
Layer 3	SoH Eligibility Gating
Layer 4	Per-Pathway Economics
Layer 5	Selection, Sanity Checks & Pricing

Two design principles run through all layers:

- **Eligibility filters; economics selects.** Earlier layers narrow the candidate set; the economic layer picks the winner from what remains.
- **Comparisons happen only in contested bands.** At SoH > 75%, Reuse dominates so heavily that full recycle math is skipped (with a cheap floor-check). At SoH < 50%, only Recycle is eligible. The middle band is where the real comparison lives.

Layer 0 — Intake & Validation

Generate trace_id (format: TRC-YYYY-NNNN), assign QR if not already assigned, capture intake_timestamp and inspector_id.

Load three input bundles:

- **Technical** (BMS / Module 1 / Entroview output)
- **Business config** (admin-configurable, versioned)
- **Market data** (live or cached, freshness-stamped)

Validate: required fields present, ranges plausible, market data freshness < 24h, config version pinned. If invalid → exception queue.

Layer 1 — Physical Damage Scoring

Three weighted tests against established standards. Replaces simple boolean safety flags with a graded, reproducible score.

Test	Standard	Score 0	Score 1	Score 2	Score 3	Weight
Visual Integrity	Suzuki IDIS, ISO 12405-4	Pristine	Minor scratches	Moderate dents	Cracks/punctures	0.40
Leakage	DOT Special Provision 188	None	Trace residue	Visible wetness	Active leak	0.35
Thermal (intake)	EUCAR / Suzuki	<40°C stable	40–50°C	>50°C	Unstable / smoke	0.25

`Damage_Score = 0.4 × Visual + 0.35 × Leakage + 0.25 × Thermal`

Damage Score	Routing
0.0 – 1.5	Safe for all pathways
1.6 – 2.5	Eligible: {REFURBISH, RECYCLE}
> 2.5	Force RECYCLE

Note on swelling: captured under Visual Integrity (a visibly deformed pack scores 2 or 3). Inspectors must be trained on the rubric for inter-rater consistency.

Layer 2 — BMS Safety Assessment

Captures defects not visible externally: internal cell failures, IR growth, voltage imbalance. Operates on Module 1 / Entroview output.

```
IF entropy_anomalies_count > 3
  OR temperature_history_max_c > 55
  OR ir_imbalance_ratio > 1.5:
  eligible n= {RECYCLE}

ELSE IF voltage_imbalance_mv > 150:
  eligible n= {REFURBISH, RECYCLE}
  # high imbalance is fixable by refurb (cell rebalancing/replacement)
```

```
# but not safe to ship as-is for reuse  
  
ELSE:  
    keep eligible from Layer 1
```

Why high voltage imbalance is not a force-recycle: cell rebalancing and weak-cell replacement is exactly what refurbishment does. It just blocks reuse, since you would not ship an imbalanced pack to a customer.

Also captured here for downstream use:

- soh_nominal, soh_confidence_low, soh_confidence_high
- cycle_count_estimate, degradation_mode, remaining_useful_life_months (RUL)
- entropy_anomalies_count → drives weak_cell_fraction directly (no heuristic estimate needed)

Confidence guard: if $\text{soh_confidence_high} - \text{soh_confidence_low} > 0.10$, flag for re-test before quoting Reuse. Wide bands suggest the SoH estimate is unreliable.

Layer 3 — SoH Eligibility Gating

```
SoH > 75%      → eligible n= {REUSE, REFURBISH, RECYCLE}  
50% < SoH ≤ 75% → eligible n= {REFURBISH, RECYCLE}  
SoH ≤ 50%      → eligible n= {RECYCLE}
```

Additional eligibility filters:

```
IF chemistry == "unknown":    eligible = {RECYCLE}  
IF cycle_count > cycle_cap:    remove REUSE      (default cycle_cap = 3000)  
IF age_years > age_cap:       remove REUSE      (default age_cap = 8)  
IF damage_score > 1.5:        REUSE already removed by Layer 1
```

Fast-path optimization: when $\text{SoH} > 75\%$ AND no Layer 1/2 issues AND chemistry is known, evaluate Reuse first. Compute a cheap recycle floor ($\text{mass} \times \text{estimated metal value} \times \text{recovery}$). If $\text{Reuse_Net} \geq 1.5 \times \text{recycle floor}$, skip full Recycle calculation. This handles the common case efficiently while still protecting the rare edge case where market conditions invert the usual ordering.

Layer 4 — Per-Pathway Economics

For each eligible pathway, compute Revenue, Costs, and Net_Value.

4.1 Configurable Inputs Catalogue

Battery-side (technical, from BMS / Module 1)

- SoH (%)
- chemistry (LFP / NMC622 / NMC811 / LCO / etc.)
- capacity_kWh, weight_kg
- age_years, cycle_count, RUL_months
- damage_score (from Layer 1)

- BMS health metrics (entropy_anomalies, ir_imbalance, voltage_imbalance, temp_max)

Business config (admin-configurable, versioned)

- processing_rate_per_kg — intake handling, sorting, basic diagnostics (applies to ALL pathways — sunk before pathway decision)
- qa_rate_per_kg — post-decision testing/certification (Reuse: full certification; Refurbish: post-rebuild verification; Recycle: zero)
- refurb_labor_rate_per_kg — default ₹180/kg
- cell_replacement_rate — ₹/cell
- hydromet_rate_per_kg, refining_rate_pct
- overhead_rate_pct, logistics_rate_per_km
- recovery_efficiency[metal] — Li, Co, Ni, Mn, Cu, Al
- yield_loss_pct — overall recycle process loss
- chemistry_composition[chemistry][metal] — % by mass
- margin_tiers — {aggressive: 30%, standard: 20%, generous: 10%}
- supplier_margin_override — per-supplier tier

Market data (live or cached)

- metal_price[metal] in ₹/kg (LME + Fast Markets, FX-converted, freshness-stamped)
- second_life_rate_per_kWh — typical ₹8,000–15,000
- refurb_pack_rate_per_kWh — typical 40–60% of new pack price

4.2 Unique Parameters per Pathway

This is what each pathway specifically needs, beyond the all-encompassing inputs (weight, distance, target margin, FX).

Aspect	REUSE	REFURBISH	RECYCLE
Revenue driver	second_life_rate × usable_kWh × multipliers	refurb_pack_rate × post-refurb usable_kWh	$\Sigma(\text{metal_mass} \times \text{recovery} \times \text{LME price}) \times (1 - \text{yield_loss})$
Pathway-specific revenue inputs	second_life_rate_per_kWh chemistry_mult (application fit) RUL multiplier OR age/cycle discount	refurb_pack_rate_per_kWh soh_restoration_delta (post-refurb SoH)	metal_price[Li, Co, Ni, Mn, Cu, Al] chemistry_composition[chemistry][metal] recovery_efficiency[metal] yield_loss_pct FX rate
Pathway-specific costs	qa_rate_per_kg (full certification) flat_repackaging_fee	refurb_labor_rate_per_kg (₹180) cell_replacement_rate × entropy_anomalies_count qa_rate_per_kg (post-refurb verify)	hydromet_rate_per_kg refining_rate_pct (× revenue) no QA needed

Common costs (all paths)	processing_rate_per_kg × weight (intake, agent visit, diagnostics)	(same)	(same)
Outbound destination	Second-life buyer (telecom, ESS, solar)	Refurb buyer (small fleets, e-rickshaw market)	Refiner / metal markets
Typical turnaround	Days	Weeks	Weeks–months

4.3 Reuse Pathway

```

usable_kWh = capacity_kWh × (SoH / 100)

# RUL multiplier – primary if available, fallback otherwise (avoid double-counting)
IF RUL_months available AND trustworthy:
    revenue_multiplier = f(RUL_months)
    # ≥24mo → 1.00 ; 12–24mo → 0.85 ; <12mo → 0.65
ELSE:
    age_discount = max(0.7, 1 - age_years × 0.03)
    cycle_discount = max(0.75, 1 - cycle_count / 10000)
    revenue_multiplier = age_discount × cycle_discount

chemistry_mult = lookup(chemistry, application)
# e.g., LFP for stationary ESS = 1.10 ; NMC default = 1.00

Reuse_Revenue = usable_kWh × second_life_rate_per_kWh
                × revenue_multiplier × chemistry_mult

Reuse_Costs = weight × processing_rate_per_kg          # intake, common to all paths
              + weight × qa_rate_per_kg              # full certification for resale
              + flat_repackaging_fee
              + (distance_in + distance_out_reuse) × logistics_rate_per_km
              + Reuse_Revenue × overhead_rate_pct

Reuse_Net = Reuse_Revenue - Reuse_Costs

```

4.4 Refurbish Pathway

```

post_refurb_SoH = min(95, SoH + soh_restoration_delta)
                # default delta ≈ 15%, configurable per chemistry
refurb_usable_kWh = capacity_kWh × (post_refurb_SoH / 100)

Refurb_Revenue = refurb_usable_kWh × refurb_pack_rate_per_kWh

# Use BMS output directly – entropy_anomalies gives weak cell count
weak_cell_fraction = entropy_anomalies_count / cells_per_pack
Replacement_Cost = entropy_anomalies_count × cell_replacement_rate

Refurb_Costs = weight × processing_rate_per_kg          # intake, common to all paths
              + weight × refurb_labor_rate_per_kg
              + Replacement_Cost
              + weight × qa_rate_per_kg              # post-refurb verification
              + (distance_in + distance_out_refurb) × logistics_rate_per_km
              + Refurb_Revenue × overhead_rate_pct

```

```
Refurb_Net = Refurb_Revenue - Refurb_Costs
```

4.5 Recycle Pathway

```
FOR metal in {Li, Co, Ni, Mn, Cu, Al}:
    metal_mass      = weight × chemistry_composition[chemistry][metal]
    recovered_mass   = metal_mass × recovery_efficiency[metal]
    metal_revenue    = recovered_mass × metal_price[metal]    # FX-adjusted

Recycle_Gross_Revenue = Σ metal_revenue
Recycle_Revenue       = Recycle_Gross_Revenue × (1 - yield_loss_pct)

Recycle_Costs = weight × processing_rate_per_kg          # intake, common to all paths
               + weight × hydromet_rate_per_kg
               + Recycle_Revenue × refining_rate_pct
               + (distance_in + distance_out_recycle) × logistics_rate_per_km
               + Recycle_Revenue × overhead_rate_pct
               # no QA – pack is going to be shredded

Recycle_Net = Recycle_Revenue - Recycle_Costs
```

4.6 Cost Composition Policy (lump-sum vs component)

To resolve the "are costs given or calculated" ambiguity, each cost field accepts either form. Admin chooses per field.

Important distinction — Processing vs QA. These are two separate testing/handling activities and are easy to confuse:

- **Processing (intake):** field agent visit, transport-in, unpacking, basic diagnostics, BMS read, damage inspection. Happens **once, before** pathway evaluation. Same cost regardless of which pathway is eventually chosen, but still subtracts from every pathway's Net Value.
- **QA (post-decision):** happens **after** the pathway is chosen. Reuse needs full certification (capacity verification, leak-down, safety cert). Refurbish needs post-rebuild verification. Recycle needs none — the pack is going to be shredded.

Both are real costs and both appear in the cost stack. Processing is identical across pathways (so it doesn't change the comparison) but it does reduce absolute Net Value, which affects price bands and the HOLD/REVIEW threshold.

Cost field	If given as lump-sum (₹)	If given as component rate
Processing (intake, all paths)	Use directly — covers agent visit, intake, basic diagnostics	processing_rate_per_kg × weight
QA (post-decision)	Use directly — Reuse: high; Refurbish: medium; Recycle: zero	qa_rate_per_kg × weight (with pathway multiplier)

Logistics	Use directly — covers both legs (in + out)	$\text{logistics_rate_per_km} \times (\text{distance_in} + \text{distance_out_pathway})$
Refurbishment	Use directly — labor + cell replacement bundled	$(\text{weight} \times ₹180) + (\text{entropy_anomalies} \times \text{cell_replacement_rate})$
Refining (recycle)	Use directly	$\text{Recycle_Revenue} \times \text{refining_rate_pct}$ (default 5%)
Overhead	Use directly	$\text{Revenue} \times \text{overhead_rate_pct}$ (default 8%)

If admin gives only "processing cost" and "logistics cost" as final lump-sums, the engine treats each as the bundled all-in number for that pathway. Refining and overhead in that case are assumed to already be folded into processing cost. Confirm preference with the team lead — this is Open Question #1.

Layer 5 — Selection, Sanity Checks & Pricing

5.1 Selection (simple)

```
winner = argmax(Net_Value) over eligible pathways

# Tiebreaker: when top two are within 5%, prefer higher hierarchy
# Rationale: more sustainable, faster turnaround, lower capital intensity
# all align with REUSE > REFURBISH > RECYCLE — a weighted scoring
# system collapses to this ordering anyway, so we encode it directly.
hierarchy_priority = { REUSE: 3, REFURBISH: 2, RECYCLE: 1 }

IF |Net_Value(top1) - Net_Value(top2)| / Net_Value(top1) < 0.05:
    winner = pathway with higher hierarchy_priority
```

Why not weighted tiebreaker scoring? All three sensible weights (sustainability, turnaround speed, execution risk) point the same direction: Reuse beats Refurbish beats Recycle. A weighted formula adds knobs that produce the same answer. The simpler hierarchy rule is correct, transparent, and trivially override-able by admin if market conditions ever invert (e.g., reuse demand collapses).

5.2 Sanity Checks

```
IF winner.Net_Value < 0:
    flag = "HOLD"
    # No profitable pathway today — warehouse and re-evaluate later

IF winner.Net_Value < hurdle_rate:
    flag = "REVIEW"
    # Exception queue for human decision
```

5.3 Pricing Band — margin as % of Net Value

Margin is applied as a fraction of Net Value (procurement convention), not of revenue. This makes "a 20% margin" mean exactly what most people think it means.

```
For tier in {aggressive: 0.30, standard: 0.20, generous: 0.10}:
    Purchase_Price(tier) = Net_Value × (1 - margin_tier)

# Output band:
P_min      = Net_Value × (1 - 0.30)  # we keep 30% – minimum price offered
P_recommended = Net_Value × (1 - 0.20) # standard 20% margin
P_max      = Net_Value × (1 - 0.10)  # generous 10% – maximum price for strategic deals
```

Internal stock: skip the pricing band. Use Net_Value as recovered value, and use margin tier as a **hurdle rate** — process now if Net_Value ≥ hurdle, hold otherwise.

Per-supplier overrides: bulk or strategic suppliers can be assigned a more generous tier in their account profile. Field agent never sees the override — they see the price band the admin configured.

6. Engine Output Object

Every quote produces this structured output, logged immutably for audit and reproducibility.

```
{
  "trace_id": "TRC-2026-0001",
  "battery_id": "EVB-88421",
  "decision": {
    "pathway": "REFURBISH",
    "rationale": "Refurbish Net Value (₹5,140) exceeds Recycle (₹4,572)",
    "flags": [],
    "tiebreaker_applied": false
  },
  "economics": {
    "revenue": 12000,
    "revenue_breakdown": { "refurb_pack_resale": 12000 },
    "costs": 6860,
    "cost_breakdown": {
      "processing": 600, "refurb_labor": 2700,
      "replacement_cells": 1200, "qa": 600,
      "logistics": 800, "overhead": 960
    },
    "net_value": 5140
  },
  "pricing": {
    "p_min": 3598, "margin_at_p_min": 0.30,
    "p_recommended": 4112, "margin_at_p_recommended": 0.20,
    "p_max": 4626, "margin_at_p_max": 0.10
  },
  "alternatives": [
    { "pathway": "RECYCLE", "net_value": 4572, "delta_vs_winner": -11.0 }
  ],
  "sensitivity": [
    "If Co price rises >20%, Recycle becomes preferred."
  ],
  "audit": {
    "config_version": "v2026-04-25-r3",
    "market_snapshot_id": "MKT-20260425-0900",
    "fx_rate_usd_inr": 83.2,
    "decision_timestamp": "2026-04-25T18:27:00+05:30",
    "input_hash": "..."
  }
}
```

```
}  
}
```

7. BMS Fields — Used vs Stored

To prevent unused-field pollution in the engine signature, BMS outputs are split into two tiers.

Used in decision logic

- soh_nominal, chemistry, capacity_kWh, weight_kg
- damage_score (composite from Layer 1)
- entropy_anomalies_count (weak cell count for refurb cost)
- ir_imbalance_ratio, voltage_imbalance_mv, temperature_history_max_c (Layer 2 triggers)
- cycle_count_estimate, age_years (Layer 3 reuse caps + Reuse revenue fallback)
- remaining_useful_life_months (Reuse revenue multiplier when available)

Stored for traceability/compliance/analytics, NOT used in decision

- trace_id, battery_id, qr_code, intake_timestamp, inspector_id
- test_duration_hours, soh_confidence_low, soh_confidence_high (used only as a re-test trigger)
- soc_current, entropy_score_pack, entropy_anomalies_cell_ids
- internal_resistance_avg_mohm, degradation_mode (analytics)
- Full event lifecycle log (Collected → Tested → Processed → Certified)

8. Worked End-to-End Example

Tracing a real battery through the full graph.

Inputs

- SoH: 65%, Chemistry: NMC622, Capacity: 2.5 kWh, Weight: 15 kg
- Age: 3 yrs, Cycles: 1,800, Safety flags: none, Distance: 50 km
- entropy_anomalies_count: 5 (out of 60 cells)

Layer 1 (Damage Scoring)

Visual: 0, Leakage: 0, Thermal: 0 → Damage Score 0.0 → safe for all pathways

Layer 2 (BMS Safety)

entropy_anomalies = 5 (> 3) → eligible ∩ = {RECYCLE}

Wait — at 5 anomalies, BMS forces recycle. Let's revise the example to show a richer comparison: assume entropy_anomalies = 3 (boundary), all other gates pass.

Layer 3 (SoH gate)

SoH 65% → eligible = {REFURBISH, RECYCLE}

Layer 4A — Refurbish

$\text{post_refurb_SoH} = \min(95, 65 + 15) = 80\%$
 $\text{refurb_usable_kWh} = 2.5 \times 0.80 = 2.0 \text{ kWh}$
 $\text{Refurb_Revenue} = 2.0 \times ₹6,000/\text{kWh} = ₹12,000$

Costs:

Processing (intake): $15 \times 40 = ₹600$
 Refurb labor: $15 \times 180 = ₹2,700$
 Cell replacement: $3 \times 400 = ₹1,200$
 QA (post-refurb): $15 \times 40 = ₹600$
 Logistics: $100 \times 8 = ₹800$
 Overhead: $12,000 \times 0.08 = ₹960$
 Total: ₹6,860

$\text{Refurb_Net} = 12,000 - 6,860 = ₹5,140$

Layer 4B — Recycle

Metal masses (NMC622 composition $\times 15 \text{ kg} \times \text{recovery} \times ₹/\text{kg}$):

Li: $1.05 \times 0.85 \times 1,200 = ₹1,071$
 Co: $0.75 \times 0.95 \times 2,800 = ₹1,995$
 Ni: $2.25 \times 0.95 \times 1,500 = ₹3,206$
 Mn: $0.75 \times 0.90 \times 200 = ₹135$
 Cu: $1.80 \times 0.95 \times 850 = ₹1,453$
 Al: $2.25 \times 0.92 \times 220 = ₹455$
 Gross: ₹8,315

After 5% yield loss: ₹7,899

Costs:

Processing (intake): $15 \times 40 = ₹600$
 Hydromet: $15 \times 60 = ₹900$
 Refining: $7,899 \times 0.05 = ₹395$
 Logistics: $100 \times 8 = ₹800$
 Overhead: $7,899 \times 0.08 = ₹632$
 Total: ₹3,327

$\text{Recycle_Net} = 7,899 - 3,327 = ₹4,572$

Layer 5 — Selection

Refurbish ₹5,140 vs Recycle ₹4,572

Difference: 11.0% (above 5% threshold, no tiebreaker needed)

Winner: REFURBISH

Pricing band (margin as % of Net Value):

$P_{\min} = 5,140 \times (1 - 0.30) = ₹3,598$ (we keep 30%)
 $P_{\text{recommended}} = 5,140 \times (1 - 0.20) = ₹4,112$ (standard)
 $P_{\max} = 5,140 \times (1 - 0.10) = ₹4,626$ (strategic supplier)

9. Open Questions for the Team Lead

- Cost composition:** Are processing/logistics/refurb/etc. given as final lump-sums, or as component rates we build up? Engine supports both, but admin needs to pick a default.
- Sourcing scope:** Warehouse-only for v1, or do we include external supplier procurement? Affects price band display vs hurdle-rate display.

7. **Module 1 contract:** Exact field names, types, freshness expectations for the BMS handoff. We need a fixed schema before integration.
8. **Vulnerable input defaults:** Who provides initial values for second-life rate, refurb rate, hydromet rate, refining rate? Operations or commercial team?
9. **Margin authority:** Who can override margin tier per supplier, and through what workflow?
10. **Damage scoring training:** Will inspectors be trained on the 0–3 rubric? Inter-rater consistency matters for the score to be meaningful.
11. **RUL availability:** Does Entroview reliably output RUL? If not, age/cycle fallback is the default.
12. **FX & quote expiry:** What's the policy on quote validity (24/48/72 hours)? Lock FX at quote time or float until purchase?

10. Build Sequence (3-person team)

Person A — Engine Core

- Layers 1–5 as pure functions (input JSON → output JSON, no UI)
- Cost composition logic (lump-sum + component-rate)
- Selection + tiebreaker + sanity checks
- Test harness with 15+ synthetic cases (every SoH band, chemistry, edge case)

Person B — Data & Integration

- Reference tables: chemistry composition, recovery efficiencies, rate cards, defaults
- Mocked market data feed (live API later)
- Engine ↔ BMS schema mapping
- Margin tier + supplier override logic

Person C — Wireframes & Flow Design

- Intake / scan screen (QR + manual fallback)
- Damage scoring screen (3-test rubric, inspector-facing)
- Decision result screen (matches output object 1:1)
- Admin config panel (versioned)
- Inventory + traceability views
- QR schema + scan-to-quote flow

Sync every 2–3 days. Schemas locked first, then layers built in parallel, integrated at the end.

11. Change Log (vs. earlier drafts)

Aspect	Earlier	Final	Why
--------	---------	-------	-----

Safety check	Boolean flags	Weighted damage score (Layer 1) + BMS gate (Layer 2)	Standard-aligned, nuanced
SoH reuse threshold	70%	75%	Industry-standard, conservative
Margin formulation	% of Revenue	% of Net Value	Procurement convention
Selection metric	Purchase Price ceiling	Net Value	Pathway choice independent of margin
RUL handling	Stacked with age/cycle	Primary if available, else fallback	Avoid double-counting
Cost composition	Built-up only	Lump-sum or component	Flexibility per admin
Architecture	Single flow	6 explicit layers	Cleaner gates, better debugging
Tiebreaker	Weighted score (configurable)	Hierarchy priority (Reuse > Refurbish > Recycle)	Weighted version collapsed to same answer; simpler is better
Margin tiers	5 tiers	3 tiers (10/20/30%)	Simpler, sufficient
BMS fields	All flow into engine	Used vs stored split	Avoid signature pollution
Weak cell estimate	Heuristic from SoH	Direct from entropy_anomalies_count	BMS already gives the number
High-SoH economics	Compare all 3 pathways	Fast-path: Reuse + cheap recycle floor check	Save compute on obvious cases

12. Closing

This specification absorbs the strongest contributions from both prior drafts: the standards-aligned damage scoring and BMS gating from one, the chemistry-aware economics, sourcing flexibility, sensitivity analysis, and configurable cost composition from the other. It also resolves the contradictions both authors spotted in each other's work.

Recommended next steps:

13. Confirm open questions with team lead, especially cost composition and sourcing scope.
14. Lock data schemas: BatteryInput, Config, MarketPrices, DecisionOutput.
15. Begin wireframe building (Person C) in parallel with engine implementation (Person A) and reference data setup (Person B).
16. Aim for a working command-line engine demo within 2 weeks; integrate with wireframes by week 4.